

UDI 01

PT. 1.1. Creación de procesos

Autores:

Ana Riquelme

2025/2026

Licencia



CC BY-NC-SA 3.0 ES **Reconocimiento – No Comercial – Compartir Igual (by-nc-sa)** No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

CREACIÓN DE PROCESOS

1. Escribe un programa en Java que cree un proceso para abrir LibreOffice Writer desde el terminal.
 - Usa ProcessBuilder para lanzar el comando libreoffice --writer.
 - Muestra por pantalla el PID del proceso creado.
 - Comprueba que el proceso se ha iniciado correctamente usando `ps aux | grep 'libreoffice'`.
2. Crea un programa en Java que inicie un proceso para ejecutar el comando gedit (editor de texto) desde Linux.
 - Muestra por pantalla el PID del proceso.
 - Espera a que el proceso termine y muestra un mensaje indicando que se ha cerrado.
 - ¿Qué debes hacer para que se cierre el proceso?
 - Comenta el código y abre varias instancias, observa los PID distintos y cierra esos procesos.

CREACIÓN Y FINALIZACIÓN DE PROCESOS

3. Escribe un programa en Java que:
 - Cree un proceso que ejecute `ping -c 100 google.com`.
 - Muestre por pantalla el PID del proceso.
 - Espere 5 segundos.
 - Termine el proceso de forma controlada usando `process.destroy()` o `process.destroyForcibly()`.
 - Comprueba que el proceso ha sido destruido usando `ps -aux | grep 'ping'`.
4. Reescribe el programa anterior de manera que se mida la diferencia en milisegundos entre el uso de `destroy()` y `destroyForcibly()`. Durante la ejecución deben cumplirse los siguientes pasos y almacenarse los tiempos en variables:
 - I. Guardar el tiempo de inicio del proceso.
 - II. Guardar el tiempo en que se inicia la finalización del proceso (al llamar a `destroy()`).
 - III. Hacer que el programa espere 2 segundos para verificar si el proceso ha finalizado.
 - IV. Calcular la diferencia de tiempo entre:
 - el instante en que se pidió la finalización con `destroy()`, y
 - el momento en que el proceso realmente terminó (si es que lo hizo).
 - V. Si después de la espera el proceso no ha finalizado, forzar su terminación con `destroyForcibly()`.
 - VI. Mostrar:
 - El tiempo que tardó en completarse la llamada a `destroy()`
 - El tiempo que tardó `destroyForcibly()`, en caso de ser necesario.
 - El tiempo total de ejecución del proceso desde que inició hasta que terminó.